



DD2395 Computer Security

DD2391 Cybersecurity Overview

Exam 2023-10-23 14:00–18:00

The problems are *not* sorted according to any perceived difficulty.

Students are allowed to bring any course materials in physical form including slides, course books, past exams, and own notes.

The exam grade is computed as follows:

- E: 35-44 points
- D: 45-54 points
- C: 55-64 points
- B: 65-74 points
- A: 75-100 points

1 Authentication - Password Storage [14 points]

Alice is building a website and she is trying out different schemes to securely store users' passwords. Bob is an attacker who can do some amount of offline computation before he steals the passwords' file, and some amount of online computation after stealing the passwords' file.

The exercise is subject to the following technical details and assumptions:

- Each user has a unique username, but different users may have the same password.
- Bob knows the list of users registered on Alice's website.
- Alice has at most 100 users using her website with passwords between 5 –10 letters.
- Bob's dictionary contains all possible user passwords.
- Bob can do K online computations and $100K$ offline computations, where K is the number of words in the dictionary.
- Slow hash functions take 100 computations per hash, while fast hash functions take 1 computation per hash.
- H_s and H_f , denote slow and fast hash functions, respectively, and Enc , an encryption algorithm, e.g., AES.
- usr and pwd denote a user's username and password, and k is an encryption key known only by Alice.

- 1a [4 points]** Calculate: (1) how many times could Bob hash every word in the dictionary using the slow hash function H_s with offline computation; (2) how many times could Bob hash every word in the dictionary using the fast hash function H_f with online computation; (3) how many times could Bob hash every word in the dictionary using slow hash function H_s with online computation. Justify your answer in all three cases.

Solution The correct answer for case (1) and (2) is 1. To evaluate a slow hash function takes 100 computations, hence for a dictionary of K words, it will take $100K$ computations, which is the exact amount of Bob's offline computation. To evaluate a fast hash function takes 1 computation, hence for a dictionary of K words, it will take K computations which is the exact amount of Bob's online computation. For case (3), Bob cannot hash the entire dictionary. Hashing the entire dictionary with the slow hash function takes $100K$ computations but Bob only has K online computation.

- 1b [6 points]** Alice is considering to use the following two password storage schemes (where $\parallel$ denotes concatenation):

1. $H_f(\text{pwd} \parallel \text{"Alice"})$
2. $(H_s(\text{pwd} \parallel \text{"Alice"}), \text{Enc}(k, H_f(\text{pwd})))$

However, Alice is worried whether Bob can: (a) learn whether two users have the same password with only online computation; and (b) learn every user's password with both offline and online computation. Help Alice by justifying your answers to her concerns in (a) and (b). You can assume that Bob knows each scheme.

Solution Scheme 1: Since the salt is fixed to "Alice", Bob can do a full run of the dictionary with online computation to learn each user's password, thus succeeding on both (a) and (b).

Scheme 2: Because of the slow hash, Bob can no longer do a full run through of the dictionary using online computation. However, he can do so using offline computation since the salt is the same for all passwords. Hence, Bob succeeds in learning every user's password.

Since encryptions are unsalted, Bob can learn if two users have the same password by comparing cyphertexts, which requires no computation. Hence, Bob succeeds on both (a) and (b).

- 1c [4 points]** Alice is now considering two additional password storage schemes:

1. $(H_f(\text{pwd} \parallel \text{usr}), \text{Enc}(k, \text{usr} \parallel H_f(\text{pwd})))$
2. $(H_s(\text{pwd} \parallel \text{usr}), \text{Enc}(k, H_s(\text{pwd})))$

However, Alice is worried whether Bob can: (a) learn whether two users have the same password with only online computation; and (b) learn every user's password with both offline and online computation. Help Alice by justifying your answers to concern (a) and (b). You can assume that Bob knows each scheme.

Solution Scheme 1: Because the salt is now different, Bob only has enough online computation to brute force a single password, hence he cannot learn whether two users have the same password. However, using offline computation he can still learn all the passwords since he can brute force the dictionary 100 times.

Scheme 2: Bob only has enough total computation to learn a single user's password, hence he cannot succeed on (b). However, since encryptions are unsalted, Bob can learn if two users have the same password by comparing cyphertexts, which requires no computation.

Grading strategy

- 1a) 2 points for the answer to the first case with its explanation; 1 point for case 2; 1 point for the case 3.
- 1b) For each scheme: 1 point for answering Yes/No correctly and 2 points for correct explanation.
- 1c) For each scheme: 1 point for answering Yes/No correctly and 1 points for correct explanation.

2 Multi Level Security [12 points]

You are tasked with the assignment to revise and complete the Multi-Level Security, Bell-LaPadula (BLP), model designed by an intern of company X. The company operates on documents of three levels of sensitivity: Top Secret (TS), Secret (S), and Public (P) such that $P \sqsubseteq S \sqsubseteq TS$, in the areas of National Security (NS) and Cyber Security (CS).

2a [6 points] Recall that, in the BLP model, labels L are pairs (SL, C) of sensitivity levels SL and compartments (subsets of areas) C , and they are used to enforce the "no read up" and "no write down" rules. In an attempt to implement the label partial order, the intern has proposed the following rules (where $*$ denotes any compartment), $(P, *) \sqsubseteq (TS, *)$ and $(P, *) \sqsubseteq (S, *)$.

1. Provide a counterexample of a security label for a document $D1$ and for a subject $S1$ for which the intern's rule incorrectly rejects (prevents) access for $S1$ on $D1$. Otherwise, if this is not possible, justify the correctness of intern's rule with respect to BLP rules.
2. Provide a counterexample of a security label for a document $D2$ and for a subject $S2$ for which the intern's rule incorrectly grants (allows) access for $S2$ on $D2$. Otherwise, if this is not possible, justify the correctness of intern's rule with respect to BLP rules.

Solution Counterexamples target the "no read up" rule. For 1, we can have $L(S1) = (TS, \{CS\})$ and $L(D1) = (S, \{CS\})$, which is accepted by BLP, but it is rejected by the intern's rule. For 2, we can have $L(S1) = (TS, \{CS\})$ and $L(D1) = (S, \{CS, NS\})$, which is rejected by BLP, but it is rejected by the intern's rule.

2b [6 points] Define the complete label model with the ordering relations and write out the label partial order that meets requirements of company X.

Solution A drawing of the subset partial order such that for any $L1 = (SL1, C1)$ and $L2 = (SL2, C2)$, we have that $L1 \sqsubseteq L2$ iff $SL1 \sqsubseteq SL2$ and $C1 \subseteq C2$, where Ci is a subset of $\{TS, S, P\}$.

Grading strategy

- 2a) 3 points for answering correctly each question. The symbol $*$ is sometimes confused as "the same" set, yet we accepted the answer if the reasoning is correct under this assumption.
- 2b) We penalize by 1 point if the drawing is missing some relations, yet the reasoning is correct.

- 3 Web Security [12 points]** Bank X learned about the CSRF vulnerability on their website. They hired an "expert" who advised them to add a random CSRF token to the sensitive `/transfer` request. A valid request now looks as follows:

```
https://x.com/transfer?to=alice&amount=1000&token=<random>
```

The CSRF token is chosen randomly and it is unique for each user. Bob, the attacker, starts looking at the welcome page of the bank and loads following URL in his browser:

```
https://x.com/welcome?name=<script>alert("Yay!");</script>
```

When this page loaded, Bob sees an alert pop up that says "Yay!".

- 3a [6 points]** Explain what kind of attack is the welcome page vulnerable to by providing the category of the attack. Bob plans to use this vulnerability to bypass the CSRF token defense. He will replace the `alert("Yay!");` with some carefully crafted JavaScript. Explain what should this JavaScript code do (it is sufficient to only explain the steps needed to steal the token).

Solution The welcome page is vulnerable to a Reflected XSS attack.

Bob could take the following steps: Load a payment form, extract the CSRF token, and then submit a transfer request with that CSRF token. Alternatively: Load a payment form, extract the CSRF token, and send it to Bob.

- 3b [6 points]** The website `x.com` now sets `SameSite=strict` for all of its cookies. Explain whether this change will stop the attack from the previous point. You can assume that the welcome page does not require a user to be logged in.

Solution The change will not stop the malicious request because it is sent from the welcome page of `x.com` which is of the correct origin domain.

Grading strategy

- 3a) 3 points if stated that website is vulnerable to Reflective XSS, otherwise 2 points if only XSS is mentioned. For the second part, 1 point for mentioning that csrf token is extracted from form, 1 point for mentioning the exact form (transfer/payment), 1 point for mentioning that the token is used to perform transfer using POST.
- 3b) 2 points for saying that this won't prevent the attack, and 4 points for reasoning that the `x.com/transfer` has the same domain as `x.com/welcome`.

4 Hash functions. [12 points] (points distributed according to analyzed requirements)

Let $x = x_1 \dots x_n$ be an input of n blocks of 256 bits. Let $H(x) = SHA(x_1) + \dots + SHA(x_n)$, where $+$ is addition module 2^{256} and SHA the Secure Hash Algorithm 2. Is H a secure hash function? For each requirement of secure hash functions, disprove the requirement or argue why the requirement is met.

Solution

- It is not weak collision resistant, since given an input of $n > 1$ blocks, every permutation of the blocks produces the same hash value.
- It is not one way. This is an algorithm to find an input x such that $H(x) = y$.
 1. generate a random block x^1 and compute $H(x^1) = h^1$ check if h^1 and 2^{256} (i.e. h^1 is odd) are coprime otherwise repeat. Probability of success $1/2$
 2. It means that there is a modular multiplicative inverse n for h^1 modulo 2^{256} : i.e., $n * h^1 \% 2^{256} = 1$ You can find the modular multiplicative inverse of h^1 modulo 2^{256} using the extended Euclidean algorithm. This is $O(\log(h^1))$.
 3. Then the hash of $x = x^1 \dots x^1$, where x^1 is repeated n -times, is $\sum_{n \text{ times}} SHA(x^1) \% 2^{256} = \sum_{n \text{ times}} h^1 \% 2^{256} = \sum_h 1 \% 2^{256} = h$.

Grading strategy

- 2 points if it is clear that student has used right definition of weak collision resistance (or if they call it just collision resistance)
- 2 points if it is clear that student has used right definition one way
- 1 point if the student has used the right definition of strong collision resistance
- 3 points if they disprove weak collision resistance
- 2 points if they make a reasonable argument (even if the assumptions on modulo arithmetic are wrong) to disprove one-way
- 1 point if they correctly disprove one-way
- 1 point if they disprove strong collision resistance

5 Protocols [14 points]

Two parties A and B want to use the following vulnerable protocol ① over a reliable channel to agree on a meeting time, wherein “X -> Y: msg” means party X sends message msg to Y:

1. B -> A: “B is asking for a meeting time”
2. A -> B: “Hi B, can you meet on <date> at <time>?”
3. B -> A: “Yes” or “No”

5a [2 points] Apart from lack of integrity and confidentiality, protocol ① is vulnerable to a replay attack by an eavesdropping attacker. For message 2 explain which of the three protocol design principles freshness, explicitness, and authenticated fields are violated. Assume A and B have agreed on a private shared key K1. Suggest how message 2 should be fixed, to ensure the principles.

Solution

- Explicitness: requires adding at least src A, dst B, optionally the time of request
- Freshness: adding counter to message 2
- authenticate fields: add HMAC for src A, dst B (using shared key)

Grading strategy

- 2 points for correct answer, even if encryption is unnecessarily included
- 2 points if everything is "correct" but a signature scheme is used for auth
- 1 point if all design principles are discussed without (correct) fix
- 1 point if one/two principle discussed perfectly, including fix (even if an extra assumption is made)
- 1 point if addresses some principles, missing to authenticate encryption

The protocol ① may be inefficient if A would multiple times query dates for which B is unavailable. To speed up the process A and B decide to follow an improved protocol ②. A can ask B for their preference by multiple choices (k-many) at once. We assume that **only A knows the public key pk** of B's RSA keypair (pk,sk) for the RSA modulus n:

1. B -> A: "B is asking for k-many suggestions for meeting times"
2. A -> B: "Hi B, which of the k choices do you prefer? option 1: <date> at <time>, option 2: <date> at <time>, ..., option k: <date> at <time>"
3. B -> A: $E(sk, l)$, where $E(sk, l) = l^{sk} \pmod n$ for $1 \leq l \leq k$, or $l = 0$ if none of the suggested times work for B.

5b [2 points] If B would instead answer both l and $E(sk, l)$, answer l can be observed. Explain what an attacker that observes full message exchanges of protocol ② can learn about B's choice, stating any assumptions.

Solution As $0^{sk} = 0$ protocol ② leaks whenever B is not available. An attacker that observes further messages from A to B, could infer that the attempted time agreement in earlier messages had been unsuccessful.

Grading strategy

- 2 points for noting leak of 0 and 1, correlating earlier messages in some way (physical observation)
- 1 point if describing correctly that observing 0 (or 1) gives specific information away but without explaining why.
- 0 points if reasoning from assumption that attacker can read everything

5c [4 points] Protocol ② is vulnerable to replay attacks due to the use of RSA encryption. Assume an attacker observes the three messages $E(sk,1)$ $E(sk,2)$ and $E(sk,3)$ during several protocol runs. Discuss which choices this attacker could confirm for B.

Solution Due to multiplicativity (and the laws of exponentiation $a^r \cdot b^r = (a \cdot b)^r$ and $(a^s)^r = (a^r)^s = a^{rs}$), the attacker can produce any of the answers $2^i 3^j$ for any $i, j \geq 0$. The confirmation of choice 1 is only possible, because $E(sk, 1)$ is given.

Grading strategy

- 3 points for correct idea, but missing some values or being unclear about values
- 2 points for correct idea, but no values/calculation provided
- 1 point only simple replay

Let protocol ③ be the previous protocol, with a different second message: A adds a unique counter i to each question in protocol ② and encrypts with a session key K .

2. A $\rightarrow$ B: sends $E(pk, K)$ and c with $c = \text{CBC}(K, \text{msg})$ and $\text{msg} = \text{"Meeting agreement (question number } i\text{): Which of the } k \text{ choices do you prefer? option 1: <date> at <time>, option 2: <date> at <time>, ..., option } k\text{: <date> at <time>"}$

5d [2 points] Change message 3 of protocol ③ so that B's answer is sent while guaranteeing integrity and confidentiality. Clearly state which cryptographic primitives you assume.

Solution Encrypt using the session key and HMAC. (Potentially send another key (encrypted) that is used for HMAC.)

Grading strategy

- 2 points for correct principles and message
- 1 point for only integrity or confidentiality
- 1 point for both, but using unkeyed hash
- 1 point for for correct message but missing principles (unless earlier answers indicate understanding that encryption provides confidentiality and MAC integrity)
- 1 point for for correct principles but missing spelling out the message

5e [4 points] The protocol is vulnerable to DoS attacks. Explain an attack and clearly state the target. Suggest a countermeasure and explain the limitations.

Solution

1. Amplification attack: As the fields of the first message are not authenticated, an attacker may send various message 1 requests, with a large k for amplification.
 1. E -> A: "B is asking for 100,000 suggestions for meeting times" countermeasures: restricting k, or authenticating the destination
2. Second attack: Resource exhaustion at A when initiating a bunch of requests, because A has to generate a session key and do cryptography. Countermeasures: authenticating src so it cannot be spoofed

Grading strategy

- 4 points for correct attack with further assumptions (like IP and a certain topology)
- 3 points for correct dos attack and countermeasure, lack/wrong: limitation / target
- 2 points for correct attack but no/incorrect countermeasure, or incomplete attack description with countermeasure
- 1 point for for correct message but missing principles (unless earlier answers indicate understanding that encryption provides confidentiality and MAC integrity)
- 1 point for only attack description, not even a target; for purely topological discussion; for incorrect description of correct attack with correct prevention; for right direction (e.g. resource exhaustion) but incomplete/incorrect attack; for for incomplete attack description with incomplete prevention.

6 Memory safety. [12 points] The following program is not memory safe:

```
1 void clear_array(int *array, int n) {
2   for (int i=0; i<n; i++)
3     array[i] = 0;
4   free(array);
5 }

6 int check_pin() {
7   int *user_pincode = calloc(10, sizeof(int));
8   clear_array(user_pincode, 10);
9   int *secret_pincode = calloc(10, sizeof(int));
10  fread(secret_pincode, sizeof(int), 10, pwd_fd);
11  fread(user_pincode, sizeof(int), 10, input_fd);
12  for (int i=0; i<10; i++) {
13    if (user_pincode[i] != secret_pincode[i])
14      return i+1;
15  }
16  return 0;
17 }
```

6a [3 points] Identify the security problems in the program.

6b [5 points] Explain the consequences of these vulnerabilities.

6c [4 points] Explain how to fix the program or how to protect against these problems.

Solution

- **user_pincode** is freed at line 4, but it is used later at line 12. **calloc** of line 9 could return the address that has been freed at line 4. Also, **secret_pincode** is not freed, which causes a memory leak.

Grading strategy

- 2 points for explaining that there is a use after free regarding the **user_pincode**
- 1 point for explaining that **secret_pincode** is not freed
- In case **calloc** of line 9 returns the address that has been freed at line 4, both **secret_pincode** and **user_pincode** points to the same memory region after line 9. In that case the pin check always succeeds, since we are comparing the same memory regions. In particular, line 11 overwrite the pincode that has been read at line 10. Moreover, if we call this function many times, we could exhaust the heap, causing subsequent memory allocations to fail.

Grading strategy

- 3 points for explaining that **secret_pincode** and **user_pincode** can be allocated to same memory by **calloc**

- 1 point for explaining that fread of user_pincode can cause a Dos attack for unmapped memory
- 2 points for explaining that multiple invocations can cause memory exhaust
- To fix the code, we should move the **free** from line 4 and have free of the two pincodes both at line 14 and line 16. A general other mechanism is to guarantee that calloc delays the reuse of memory regions. This will not solve the problem, but will make unlikely that the allocation at line 9 reuses that memory region.

Grading strategy

- 1 point for removing free function from clear_array
- 1 point for adding free for user_pincode before returning in check_pin function
- 1 point for adding free for secret_pincode before returning in check_pin function
- 1 point if the added free are executed in all cases

- 7 Side channels. [12 points]** Suppose that a system uses a shared data cache (no instruction cache) of 256 bytes, 4 lines, 64 bytes per line. The cache is directly mapped. The array M consists of 32 entries of 4 bytes. The first sixteen entries of the array are mapped to the first cache line, the second sixteen entries are mapped to the second cache line. The variables A and B are secret numbers and C is a public variable that is known by the attacker. Variables A , B and C are all allocated in the same block of 64 bytes, which is mapped on the third cache line. Finally, assume that B and C are smaller than 32. The victim program is the following:

```
1 IF (A == 0)
2     A += M[C]
3 IF (C >= 16)
4     A += M[B]
```

An access driven attack (i.e. one that detects which cache line has been evicted) can extract some information about the secret variables of this program.

- 7a [3 points]** Explain why this program is not secure and which instructions can cause information leakage.

Solution Memory load (2) is executed only if $A == 0$. Instruction (4) accesses an address that depends on the value of B

Grading strategy

- 1 point for explaining that there is a memory load whose address is secret dependant
- 1 point for explaining that there is a memory load whose execution is secret dependant
- 1 point for identifying lines 2 and 4

- 7b [5 points]** Explain why information is leaked and which information can be leaked for different initial values of the variables.

Solution We focus only on the cache lines that contain M , since the other variables are all in the third cache line and do not collide with lines used for M :

- (Case 1) Let $A == 0$ and $C < 16$. The memory load (2) accesses cache line 0 and the second memory load is not executed. Therefore the program accesses cache lines [0]
- (Case 2) Let $A == 0$, $C \geq 16$, $B < 16$. The memory load (2) accesses cache line 1, the second memory load accesses cache line 0. Therefore the program accesses cache lines [0, 1]
- (Case 3) Let $A == 0$, $C \geq 16$, $B \geq 16$. The memory load (2) accesses cache line 1, the second memory load accesses cache line 1. Therefore the program accesses cache lines [1]
- (Case 4) Let $A \neq 0$, $C < 16$. No load from M is executed. Therefore the program accesses cache lines []
- (Case 5) Let $A \neq 0$, $C \geq 16$ and $B < 16$. Only the second load is executed and it accesses cache line 0. Therefore the program accesses cache lines [0]

- (Case 6) Let $A \neq 0$, $C \geq 16$ and $B \geq 16$. Only the second load is executed and it accesses cache line 1. Therefore the program accesses cache lines [1]

Therefore using an access driven side channel the attacker can uniquely identify cases 2 and 4. Also, if the cache lines accesses are [0], the attacker can uniquely identify cases 1 and 5. Finally, if the line accesses are [1] the attacker learns that either $A = 0$ or that $A \neq 0 \wedge B \geq 16$.

Grading strategy

- 1 point for explaining that information regarding A and B can be leaked
- 1 point for explaining that B can be leaked only if $C \geq 16$
- 1 point for explaining that only $A \neq 0$ and $B \geq 16$ is leaked
- 2 points for a more detailed analysis of what can be leaked (1 point if student does not take into account that C is known)

7c [4 points] Describe how you can fix the program or prevent the attack.

Solution The simplest solution is to first access both $M[0]$ and $M[16]$. This would prime the cache and guarantee that we always access lines [0,1] independently of A and B . Alternatively, we can clean the cache when we exit, since this will guarantee that the program does not leave a footprint on the cache.

Grading strategy

- 2 points for any reasonable solution
- 2 points for explaining why solution is correct (e.g. it is ok to say that you must flush at the beginning, but only if you assume that the attacker does not share memory with the victim)

8 Social engineering. [12 points]

Here's a mail (hypothetically) sent to all KTH employees via bcc. Anders is one of the recipients.

Subject: in 10 minutes! Re: Johan's birthday zoom today
Date: Wed, 26 Oct 2023 11:35:24 +0200
From: Lars <redacted@kth.se>
To: Lars <redacted@kth.se>

Just a quick reminder, please join the zoom, we start in 10 minutes!

----- Forwarded Message -----
Subject: Johan's birthday zoom today
Date: Wed, 26 Oct 2023 09:34:24 +0200
From: Lars <redacted@kth.se>
To: Lars <redacted@kth.se>

Hej!

As many of you know, it's Johan's birthday today, and it's a round one! He's not at the office today since he's presenting at the ACM IMC conference this week, but we've asked him to attend a zoom meeting at 11:45h and we'd like all of his friends and colleagues to join and congratulate him.

Sorry for the short notice, it's been a busy week! We know you are all busy as well, but please join at least for a few minutes if you can, we hope that everyone can at least stop by and say hello. The more we are, the nicer the surprise.

Here's the link, see you soon!
<https://kth-se.zoom.us/j/141592653>

/Lars

When Anders clicks on the link, he gets a popup error message and is asked to sign in again. He clicks on the SSO icon for single-sign on and gets redirected to <https://kth.se>, where he enters his KTH username and password as usual at kth.se. After that, he gets automatically redirected back to <https://kth-se.zoom.us/j/141592653> where he gets the message that the meeting cannot be found. He then gets another e-mail saying that the zoom had to be postponed.

8a [9 points] Identify 3 strategies (1 point each) used by the social engineer, for each identify the human trait they exploit (1 point each), and quote or mention the parts of the e-mail you refer to (1 point each).

Solution Sample answers (don't need to be in these exact words):

1. rush/urgency (reduced cognitive ability), "we start in 10 minutes!" or "see you soon!"

2. peer pressure (people want to be helpful, cooperate, be nice, not deviate from social expectations), "we'd like all of his friends and colleagues to join and congratulate him." or "we hope that everyone can at least stop by and say hello" or " The more we are, the nicer the surprise."
3. building rapport/showing empathy (people agree to more from someone they like), "We know you are all busy", or "but please join at least for a few minutes if you can," or "Sorry for the short notice, it's been a busy week!" or enthusiastic/friendly tone "it's a round one!"
4. pretext/trappings of the role (attribution, trust) by e.g. sending from KTH address, similar link as usual KTH zoom links, similar process to sign on, plausible story, plausible formality level in language, very common first name with high likelihood of presence in a division plus vagueness about the age to increase probability.

8b [1 point] What did the attacker get?

Solution The attacker got the login credentials from Anders (and anyone else that did the same).

8c [2 points] Give two examples of what the attacker can do now.

Solution The attacker can now do anything that Anders can do, e.g. access/modify resources, impersonate Anders to other people or servers, try Anders' password for Anders' accounts elsewhere, may have gotten Anders to install malware.

Grading strategy

- It follows the exact assignments of points in the exercise and considers as correct any solutions that are variations of the above.